7. Design a Content Delivery Network

Key Terms:

Term	Full Name	What it does
CDN	Content Delivery Network	Globally distributed edge servers for fast content delivery
POP	Point of Presence	Edge server location close to users
GeoDNS	Geographic DNS	DNS that returns different IPs based on user location
Anycast	Network routing	Same IP announced from multiple locations; BGP routes to nearest
TTL	Time To Live	How long content stays cached before refresh
Origin	Source server	The original server where content lives
Shield	Mid-tier cache	Regional cache layer protecting origin from traffic
Purge	Cache invalidation	Removing stale content from edge servers

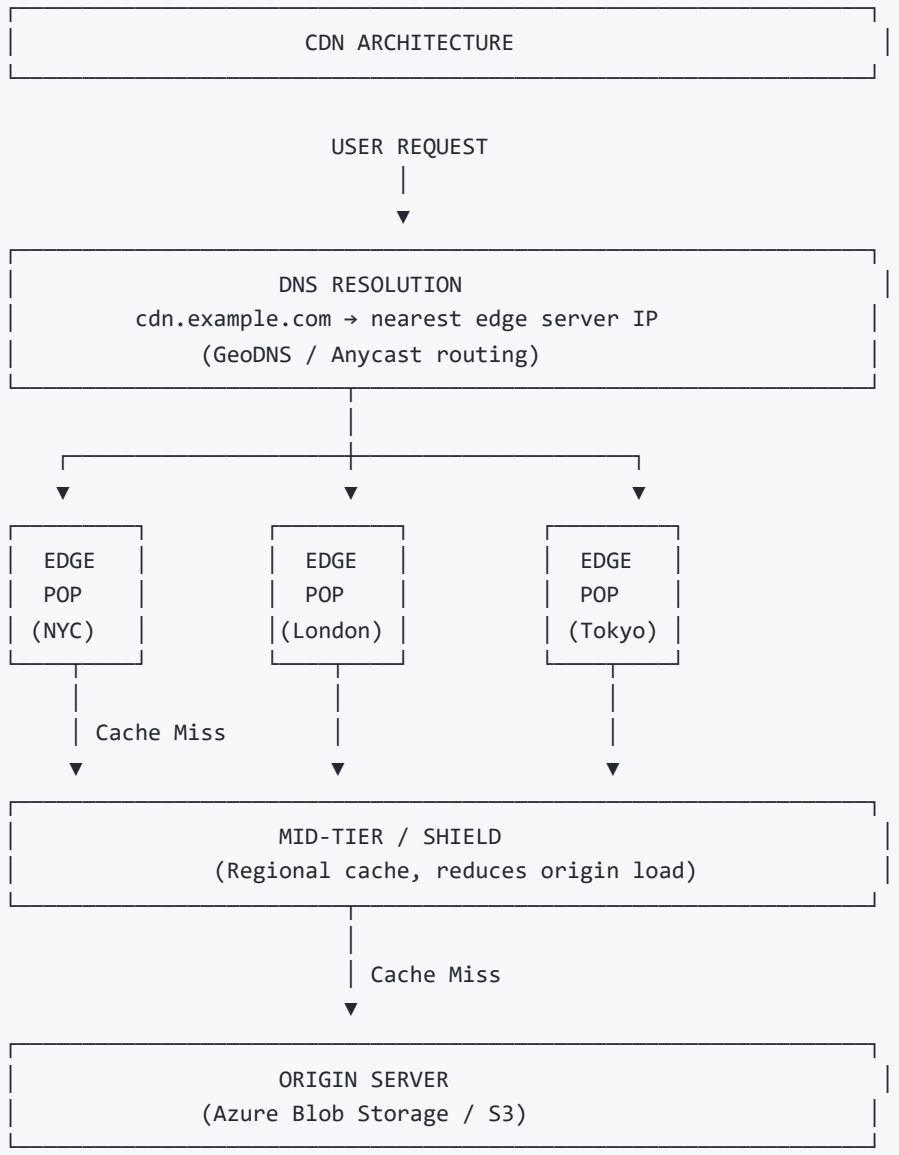
Problem Statement

Design a CDN that delivers static content (images, videos, files) with low latency to users worldwide.

Clarifying Questions to Ask

1. **Content types**: Static files, streaming video, or both?
2. **Scale**: How much data? How many requests per second?
3. **Latency target**: Max acceptable latency? (<50ms?)
4. **Cache hit ratio**: Target cache hit rate? (>95%?)
5. **Origin**: Single origin or multiple origins?
6. **Invalidation**: How quickly must cache updates propagate?
7. **Security**: DDoS protection, token auth needed?

High-Level Design



CACHE HIERARCHY:

```
┌──────────────────────────────────────────────────────┐
│  L1: Browser Cache (seconds-minutes)                 │
│  L2: Edge POP Cache (minutes-hours)                  │
│  L3: Shield/Mid-tier Cache (hours-days)              │
│  L4: Origin (permanent)                              │
└──────────────────────────────────────────────────────┘
```

API Design

Content Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/distributions`	Create a CDN distribution

`PUT`	`/api/v1/distributions/{id}`	Update distribution config
`POST`	`/api/v1/distributions/{id}/invalidate`	Invalidate cache
`GET`	`/api/v1/distributions/{id}/stats`	Get cache hit/miss stats

Cache Invalidation Request

```
POST /api/v1/distributions/{id}/invalidate
{
  "paths": ["/images/*", "/css/main.css"],
  "strategy": "immediate"   // or "soft"
}
```

Edge Request Flow

```
GET /assets/image.jpg
Headers:
  Host: cdn.example.com
  If-None-Match: "etag-abc123"

Response: 200 OK (cache miss) / 304 Not Modified (cache hit)
Headers:
  Cache-Control: public, max-age=86400
  ETag: "etag-abc123"
  X-Cache: HIT | MISS
  X-Edge-Location: us-east-1
```

Key Components

Component	Purpose	Technology
GeoDNS	Route to nearest POP	Route 53 / Azure Traffic Manager
Edge Server	Serve cached content	Nginx/Varnish
Shield	Reduce origin load	Regional cache tier
Origin	Source of truth	Blob storage
Purge Service	Invalidate cache	Message queue
Analytics	Track hits/misses	Log aggregation

Cache Key Design

```
Cache Key = Hash(URL + Vary headers + Query params)

Examples:
  /images/logo.png → key: "abc123"
  /images/logo.png?v=2 → key: "def456" (different version)
  /api/user (Accept: json) → key: "ghi789"
  /api/user (Accept: xml) → key: "jkl012" (different format)

Cache-Control Headers:
  public, max-age=86400              # Cache for 24h
  private, no-cache                   # Don't cache
  s-maxage=3600, max-age=60          # CDN: 1h, Browser: 1min
```

Cache Invalidation Strategies

```
1. TTL-based: Content expires after set time
   Pros: Simple, automatic
   Cons: Stale content until expiry

2. Purge API: Explicitly invalidate by URL/pattern
   POST /purge {"urls": ["/images/logo.png"]}
   Pros: Immediate
   Cons: Need to track what to purge

3. Versioned URLs: Append version to filename
   /images/logo.v123.png → /images/logo.v124.png
   Pros: Instant update, cache-friendly
   Cons: URL management

4. Soft Purge / Stale-While-Revalidate:
   Serve stale, fetch fresh in background
   Pros: Fast response, fresh content soon
   Cons: Brief staleness
```

Scalability Considerations

- **POPs**: Deploy edge servers in 50+ locations globally
- **Consistent hashing**: Distribute content across edge servers
- **Hot content**: Replicate popular content to all edges
- **Bandwidth**: Use tiered architecture to reduce origin bandwidth
- **DDoS protection**: Rate limiting, challenge pages at edge

Trade-offs

Decision	Option A	Option B	Recommendation
Routing	GeoDNS	Anycast	**Anycast** - faster failover
Cache storage	Memory	SSD	**Hybrid** - hot in RAM, warm on SSD
Invalidation	TTL	Purge	**Versioned URLs** - best of both
Architecture	2-tier	3-tier (shield)	**3-tier** - protects origin

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
CDN	Edge caching and content distribution	Tech Guide
Consistent Hashing	Even distribution across edge servers	Tech Guide
Load Balancer	Traffic distribution and health checks	Tech Guide

---

How to Explain It Well

> **Opening**: "A CDN reduces latency by serving content from servers close to users. I'll design for 95%+ cache hit ratio with sub-50ms latency."

> **Key talking points**:
>
> - "Anycast routing: same IP announced from all POPs, BGP routes to nearest"
> - "Shield tier absorbs cache misses, prevents origin overload"
> - "Versioned URLs for instant updates without purging"
> - "Cache key includes Vary headers to handle content negotiation"
> - "Consistent hashing distributes content across edge servers"
